

Money Transfer System – Project Report

Project: Money Transfer System (GCP Capstone)

Repository: MoneyTransferAppGCP

Date: April 15, 2026

Team: Sriram M K(SMK5), Yugash D (YD5), Vadigi Eesha Kamal (vkamal3)

Table of Contents

- 1. [Project Overview](#)
 - 2. [What the Application Does](#)
 - 3. [Technology Stack](#)
 - 4. [System Architecture](#)
 - 5. [Backend – How It Works](#)
 - 6. [Frontend – How It Works](#)
 - 7. [Google BigQuery Analytics](#)
 - 8. [Security](#)
 - 9. [Database Design](#)
 - 10. [Key Features & Business Rules](#)
 - 11. [How to Run the Project](#)
 - 12. [Project Structure](#)
 - 13. [Sample Data](#)
 - 14. [What Was Built – Summary](#)
-

1. Project Overview

The **Money Transfer System** is a full-stack banking application that allows users to securely send money between accounts, check their balance, and view their transaction history. It was built as a capstone project covering the full software development lifecycle – from writing the server logic, to building the user interface, to integrating a cloud analytics platform on Google Cloud Platform (GCP).

The project was developed progressively across five learning modules:

Module	What Was Built
Git	Version control setup, branching strategy, commit history

Module	What Was Built
Advanced Java	Core business rules – accounts, transactions, validation
Spring Boot	REST API server – the "brain" of the application
React (Vite)	User interface – the pages users interact with
Google BigQuery	Real-time analytics and business intelligence on GCP

2. What the Application Does

From a **user's perspective**, the application provides four main things:

Check Your Balance

After logging in, users are greeted by name on a dashboard that shows their current account balance, formatted in USD.

Send Money

Users can transfer funds to another account by entering the destination account number and the amount. The system validates the request, shows a review screen, and confirms the transfer. Every transfer gets a unique reference number.

View Transaction History

A paginated table shows every transfer the user has sent or received, with the date, amount, direction (Sent / Received), and whether it succeeded or failed.

View Profile

Users can see their account details – name, account ID, status, balance, and the date their account was opened.

From an **admin's perspective**, there is also a live analytics dashboard (via Google BigQuery) that shows:

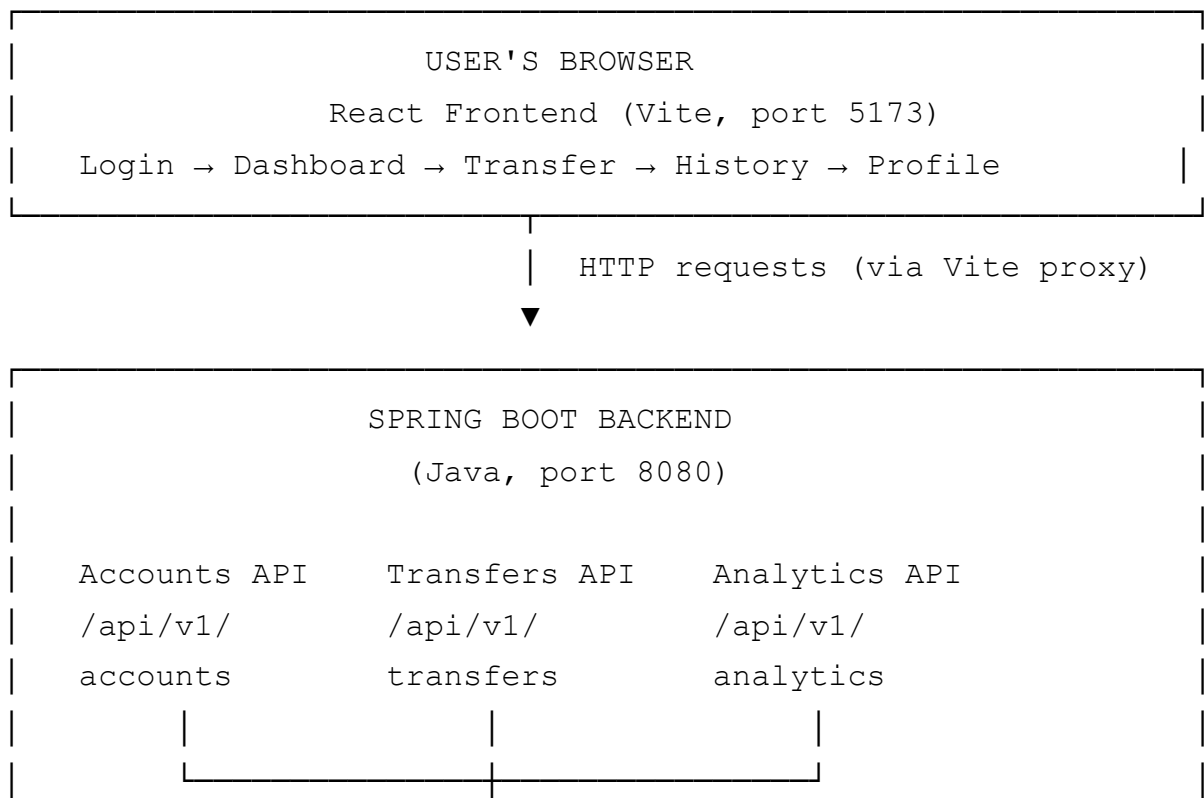
-  Daily transfer volumes and totals
-  Overall success and failure rates
-  Most active accounts
-  Peak hours for transfers
-  Summary KPIs (total volume, average transfer, etc.)

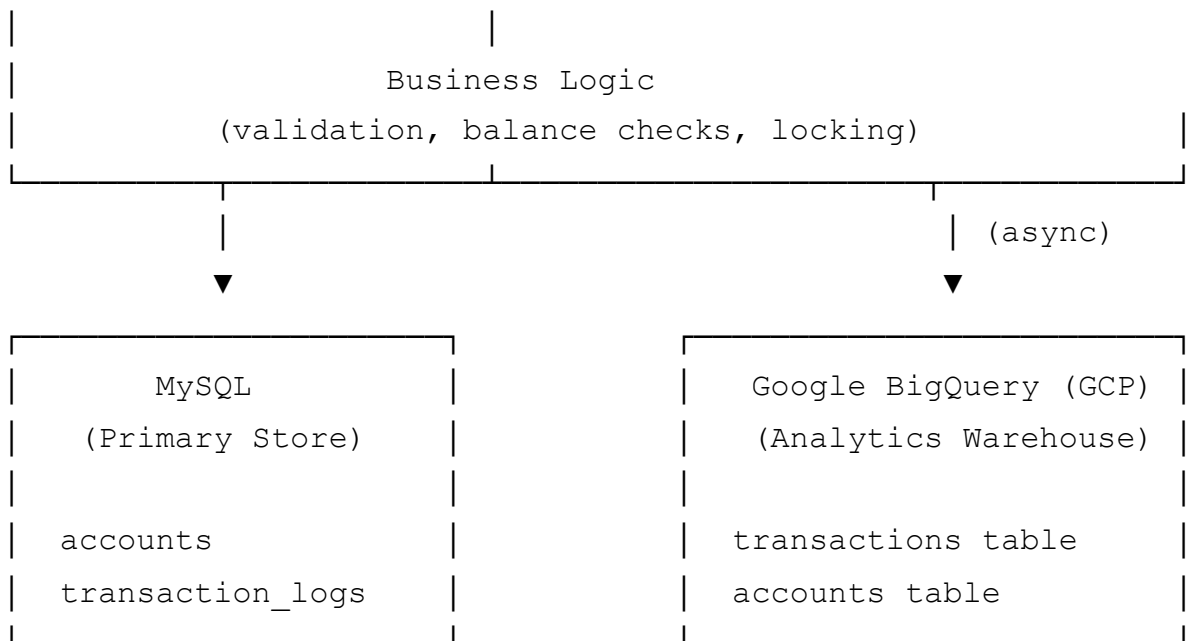
3. Technology Stack

Layer	Technology	What It Does
Backend Language	Java 17	Core application logic
Backend Framework	Spring Boot 4	Runs the REST API server
Database	MySQL 8	Stores accounts and transactions
Security	Spring Security	Protects all API endpoints with login
Cloud Analytics	Google BigQuery	Real-time analytics data warehouse
Frontend Language	JavaScript (React 19)	Builds the user interface
Frontend Build Tool	Vite 8	Runs and bundles the frontend
HTTP Client	Axios	Connects frontend to backend
Code Generation	Lombok	Reduces Java boilerplate code
Testing	JUnit 5	Automated unit tests

4. System Architecture

The system is made up of three main layers that communicate with each other:





Key point: The backend talks to both MySQL (for real banking operations) and Google BigQuery (for analytics). BigQuery updates happen in the background so they never slow down the user's experience.

5. Backend – How It Works

The backend is a REST API – it receives requests from the frontend and responds with data in JSON format. It's organised into clear layers:

Controllers – The Entry Points

There are three controllers that handle incoming requests:

Controller	Endpoints Handled
AccountController	Get account details, get balance, get transaction history
TransferController	Execute a money transfer
AnalyticsController	Query BigQuery analytics (admin only)

Services – The Business Logic

Services contain the rules that make the banking work correctly:

TransferService – handles every transfer in this exact order:

1. Checks the idempotency key (prevents the same transfer being processed twice)
2. Locks both accounts in a consistent order (prevents race conditions if two transfers happen simultaneously)

3. Validates both accounts are active
4. Deducts money from the sender's account
5. Adds money to the receiver's account
6. Saves a record of the transfer
7. Sends the transfer data to BigQuery in the background

AccountService — handles account queries: fetching account details, balance, and paginated transaction history.

BigQueryService — handles all communication with Google BigQuery: streaming transfer events in real time and running the five analytics queries.

Repositories — The Database Layer

Two repository interfaces handle all database reads and writes:

- `AccountRepository` — reads and writes account data. Uses a **database lock** when processing transfers to prevent two transfers from reading the same balance at the same time.
- `TransactionLogRepository` — reads and writes transaction history. Checks idempotency keys.

Error Handling

A single `GlobalExceptionHandler` catches all errors across the entire application and returns clean, consistent error messages:

Situation	Error Returned
Account doesn't exist	404 ACCOUNT_NOT_FOUND
Account is locked or closed	409 ACCOUNT_NOT_ACTIVE
Not enough money	409 INSUFFICIENT_BALANCE
Transfer already processed	409 DUPLICATE_TRANSFER
Invalid input data	400 VALIDATION_FAILED
BigQuery is unavailable	503 BIGQUERY_ERROR
Anything unexpected	500 INTERNAL_ERROR

6. Frontend — How It Works

The frontend is a Single Page Application (SPA) – it loads once and navigates between screens without full page reloads. It's built with React.

Pages

Page	URL	What It Shows
Login	/login	Username/password form with validation
Dashboard	/dashboard	Welcome message, balance card, quick action buttons
Transfer	/transfer	3-step transfer form (Details → Review → Confirmation)
Transaction History	/transactions	Paginated table with Sent/Received/All tabs
Profile	/profile	Account details, status badge, balance

How Login Works

There is no separate login endpoint on the backend. Instead, when a user enters their credentials, the frontend:

1. Encodes `username:password` into a Base64 string
2. Makes a test request to the backend's account endpoint using those credentials
3. If the backend says "OK" → the user is logged in and the token is saved
4. If the backend says "401 Unauthorized" → the user sees an error message

The token is stored in the browser's `localStorage` and sent with every subsequent request.

3-Step Transfer Flow

The transfer form guides users through three steps to reduce mistakes:

Step 1: Details

Enter:

- Destination ID
 - Amount
- 123

[Next →]

Step 2: Review

Confirm:

- From: Account 1
- To: Account 3

- Amount: \$500.00

[Confirm] [Back]

Step 3: Confirmation

Shows:

 Transfer complete!
Transaction ID: abc-

Amount: \$500.00

[Back to Dashboard]

A unique reference key is generated at Step 2 to ensure the transfer cannot accidentally be submitted twice.

7. Google BigQuery Analytics

BigQuery is Google's cloud data warehouse. In this project, it acts as a **live mirror** of all transfer activity, enabling powerful analytics queries without affecting the main MySQL database.

How Data Gets Into BigQuery

Every time a transfer is processed – whether it succeeds or fails – the backend immediately sends the transaction details to BigQuery **in the background** (asynchronously). This means:

- The user gets their response instantly
- BigQuery gets updated in the background
- A BigQuery outage never affects the core banking functionality

BigQuery Tables

Two tables are maintained in BigQuery under the dataset `money_transfer_analytics`:

transactions table – one row per transfer attempt:

- Transaction ID, sender account, receiver account
- Amount, status (SUCCESS/FAILED), failure reason
- Timestamp (partitioned by date for fast queries)

accounts table – balance snapshots after every transfer:

- Account ID, holder name, balance, status
- Snapshot timestamp

Analytics Endpoints

Five analytics queries are available via the REST API (admin access only):

Endpoint	What It Returns
<code>GET /api/v1/analytics/summary</code>	All KPIs in one call – totals, rates, averages
<code>GET /api/v1/analytics/daily-volume?days=30</code>	Transfer counts and totals per day
<code>GET /api/v1/analytics/success-rate</code>	Success vs failure breakdown with percentages
<code>GET /api/v1/analytics/top-accounts?limit=10</code>	Most active accounts ranked by transfer count

Endpoint	What It Returns
GET /api/v1/analytics/peak-hours?top=5	Busiest hours of the day for transfers

Enabling BigQuery

BigQuery is **off by default** so the application runs without any GCP credentials in local development. To enable it, update `application.properties` :

```
bigquery.enabled=true
bigquery.project-id=your-gcp-project-id
bigquery.credentials-file=/path/to/service-account-key.json
```

8. Security

The application uses **HTTP Basic Authentication** — every request to the API must include a username and password. All API endpoints are protected except the health check.

User Roles

User	Password	Access Level
user	password	Can view accounts and make transfers
admin	admin	All of the above + access to analytics

Access Rules

API Path	Who Can Access
/api/v1/accounts/**	All logged-in users
/api/v1/transfers	All logged-in users
/api/v1/analytics/**	Admin only
/actuator/health	Public (no login required)

Additional Protections

- **Optimistic locking** — the database prevents two updates to the same account from conflicting silently

- **Pessimistic locking** – when processing a transfer, both accounts are locked in the database so no other transfer can read a stale balance
 - **Input validation** – all incoming data is validated before any processing begins
 - **Idempotency** – every transfer has a unique key; submitting the same transfer twice is safely rejected
-

9. Database Design

The MySQL database has two tables:

accounts Table

Column	Type	Description
id	Auto-number	Unique account identifier
holder_name	Text	Full name of the account owner
balance	Decimal	Current account balance (never goes below 0)
status	Text	ACTIVE , LOCKED , or CLOSED
version	Number	Used internally to detect conflicting updates
last_updated	Timestamp	When the account was last changed

transaction_logs Table

Column	Type	Description
id	UUID text	Unique transaction identifier
from_account	Number	Account that sent the money
to_account	Number	Account that received the money
amount	Decimal	How much was transferred
status	Text	SUCCESS or FAILED
failure_reason	Text	Why it failed (if applicable)
idempotency_key	Text	Unique key to prevent duplicates
created_on	Timestamp	When the transfer was attempted

The `idempotency_key` column has a **unique constraint** – if the same key is submitted twice, the database will reject it at the lowest possible level.

10. Key Features & Business Rules

Business Rules for Transfers

Before any money moves, the system checks all of these in order:

1.  Source and destination accounts must be different
2.  Both accounts must exist
3.  Both accounts must be in `ACTIVE` status (not `LOCKED` or `CLOSED`)
4.  The transfer amount must be greater than \$0.01
5.  The sender must have enough balance to cover the amount
6.  The idempotency key must not have been used before

If any check fails, the transfer is rejected with a clear error message and nothing is changed in the database.

What Happens During a Successful Transfer

1. All rules pass 
2. Sender's balance decreases (e.g. \$10,000 → \$9,500)
3. Receiver's balance increases (e.g. \$5,000 → \$5,500)
4. A `SUCCESS` record is saved to `transaction_logs`
5. The result is sent to BigQuery in the background
6. User sees: "Transfer complete – Transaction ID: abc-123"

What Happens During a Failed Transfer

1. A rule check fails (e.g. insufficient balance)
 2. Nothing changes in the database (full rollback)
 3. A `FAILED` record is saved to `transaction_logs` (for audit)
 4. User sees: "Insufficient balance" (or the relevant error)
-

11. How to Run the Project

Prerequisites

- Java 17 or higher
- Node.js 18 or higher
- MySQL 8 running locally
- (Optional) A GCP project with BigQuery enabled

Step 1 – Set Up the Database

Create a MySQL database named `moneytransfer` :

```
CREATE DATABASE moneytransfer;
```

The tables and sample data are created automatically when the backend starts.

Step 2 – Start the Backend

```
cd backend\MoneyTransferAppGCP
.\mvnw.cmd spring-boot:run
```

The backend will be available at `http://localhost:8080` . Sample accounts and transactions are loaded automatically on first start.

Step 3 – Start the Frontend

```
cd frontend\MoneyTransferSystem\frontend
npm install
npm run dev
```

The frontend will be available at `http://localhost:5173` .

Step 4 – Log In

Open `http://localhost:5173` in your browser and log in with:

- Username: `admin` , Password: `admin`
- Username: `user` , Password: `password`

12. Project Structure

GCP capstone/

```
|
├── backend/MoneyTransferAppGCP/           ← Spring Boot REST API
|   ├── src/main/java/
|   |   └── com/example/moneytransfer/
|   |       ├── controller/               ← API entry points
|   |       |   ├── AccountController    ← Account endpoints
|   |       |   ├── TransferController   ← Transfer endpoint
|   |       |   └── AnalyticsController ← BigQuery analytics endpoints
|   |       ├── service/                 ← Business logic
|   |       |   ├── TransferService      ← Core transfer processing
|   |       |   ├── AccountService       ← Account queries
|   |       |   └── BigQueryService      ← GCP analytics
|   |       ├── domain/
|   |       └── entity/                  ← Account, TransactionLog
(database models)
|   |   ├── dto/                        ← Request/response data shapes
|   |   └── enums/                      ← AccountStatus,
TransactionStatus
|   |   ├── exception/                  ← Custom error types
|   |   ├── repository/                ← Database access layer
|   |   ├── config/                    ← Security, BigQuery setup
|   |   └── exception/                  ← Global error handler
|   ├── src/main/resources/
|   └── application.properties          ← Configuration (DB, BigQuery,
server)
|   ├── db/
|   |   ├── schema.sql                 ← Creates tables on startup
|   |   └── data.sql                   ← Loads sample accounts on
startup
|   ├── bigquery/
|   |   ├── 01_bigquery_setup.sql      ← Creates BigQuery tables
|   |   ├── 02_analytics_queries.sql   ← The 5 analytics SQL queries
|   |   └── README.md                  ← BigQuery setup guide
|
└── frontend/MoneyTransferSystem/frontend/ ← React SPA
    ├── src/
    |   ├── pages/                     ← Login, Dashboard, Transfer,
History, Profile
    |   └── components/                 ← Navbar, LoadingSpinner,
forms, tables
```



13. Sample Data

The following accounts are loaded automatically when the application starts:

Account ID	Name	Balance	Status
1	Alice Johnson	\$9,500.00	✓ ACTIVE
2	Bob Smith	\$5,300.00	✓ ACTIVE
3	Carol White	\$7,700.00	✓ ACTIVE
4	David Brown	\$2,500.00	🔒 LOCKED
5	Eve Davis	\$0.00	✗ CLOSED

Three sample transactions are also pre-loaded to demonstrate the history page:

From	To	Amount	Status
Alice → Bob	\$500.00	✓ SUCCESS	
Bob → Carol	\$200.00	✓ SUCCESS	
Alice → Carol	\$9,999.00	✗ FAILED (Insufficient balance)	

Useful test scenarios:

- **Successful transfer:** Login as `admin` (Alice), send \$500 to account `2` (Bob)
- **Insufficient balance:** Try to send more than the available balance
- **Locked account:** Try to send from account `4` (David)
- **Closed account:** Try to send from account `5` (Eve)